Java Turtle Graphics

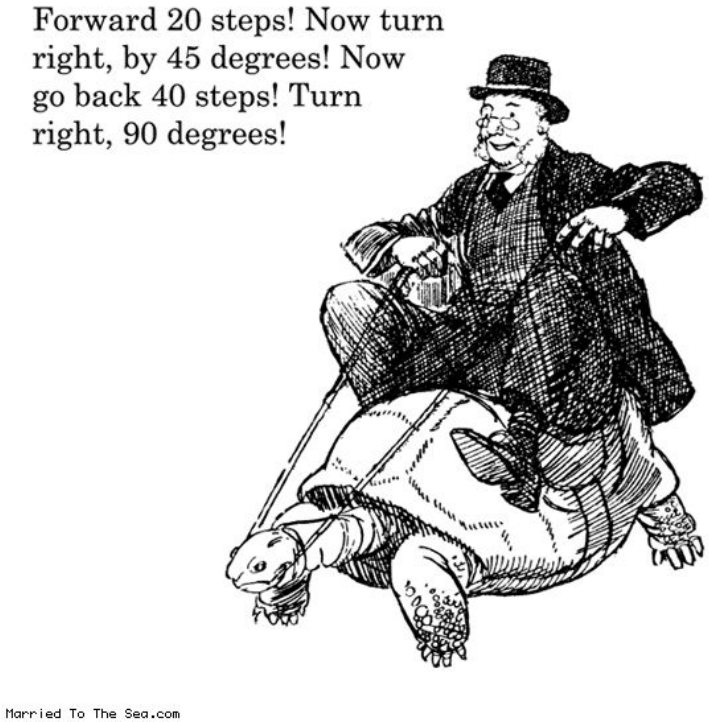
CS 141

Turtle Graphics

Draw things using a little turtle dragging a pen. Believe it or not, using a “turtle” to crawl around and drawing things made complete sense in the late 60s through the late 80s. I know, it was a different time.

Turtle has a pen and can draw while moving around. We can change the color of the pen, the thickness of the lines drawn, and so on.

Picture is worth a thousand words



http://www.marriedtothesea.com/042010/logo-version-1.gif

Some things students have done

https://docs.google.com/presentation/d/1ERG5aE7dhQTNke_0Mdq-FrSfsThGiOHmN4NBkY-1Gv0/

Show how to download and run

Also the different folders in a VS code project

Download, unzip, and run this project

Turtle Graphics outline

- DEMO running the code
- Break down example code
  - constructors
  - methods
  - application programming interface (API)
- Review Questions

Example Turtle Graphics code

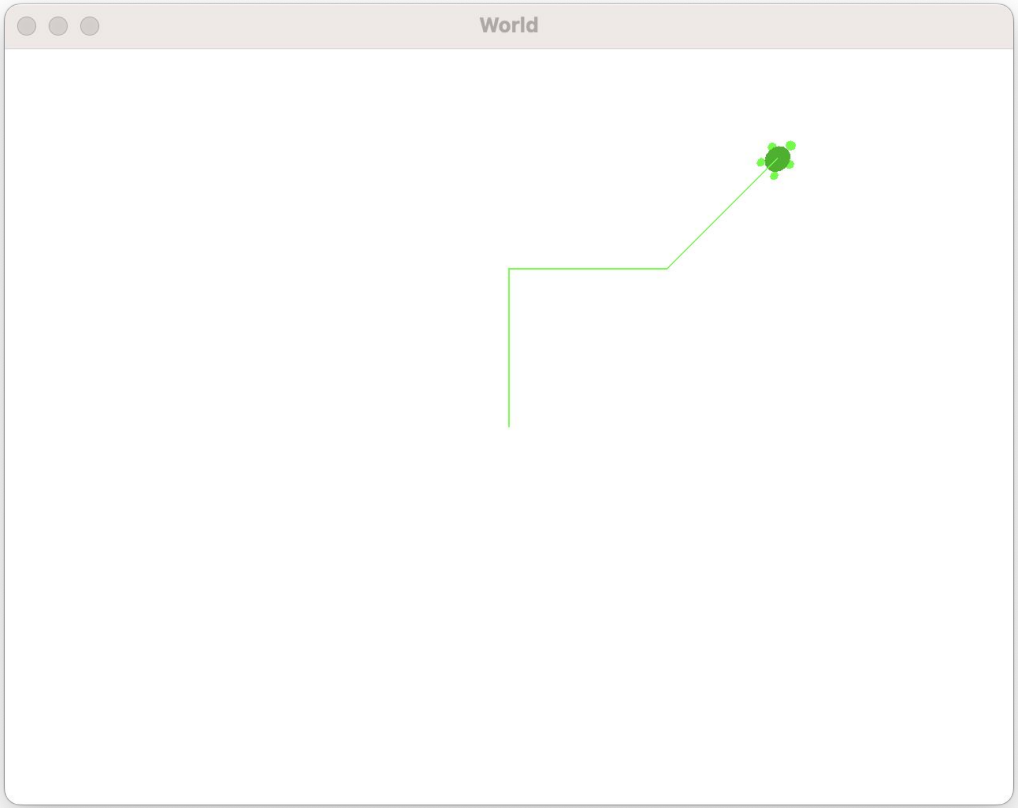
```
public class TestTurtle {
    public static void main(String[] args) {
        World world = new World();
        Turtle jamal = new Turtle(world);
        jamal.forward(100);
        jamal.turn(90);
        jamal.forward(100);
        jamal.turn(-45);
        jamal.forward(100);
    }
}
```

```
World world = new World();
Turtle jamal = new Turtle(world);
jamal.forward(100);
jamal.turn(90);
jamal.forward(100);
jamal.turn(-45);
jamal.forward(100);
```

What does this draw?

Discuss with those around you and try to figure out what it draws

```
World world = new World();
Turtle jamal = new Turtle(world);
jamal.forward(100);
jamal.turn(90);
jamal.forward(100);
jamal.turn(-45);
jamal.forward(100);
```



```
World world = new World();
Turtle jamal = new Turtle(world);
jamal.setPenWidth(10);
jamal.setPenColor(Color.BLACK);
jamal.forward(100);
jamal.turn(90);
jamal.forward(100);
jamal.turn(-45);
jamal.forward(100);
```

That's hard to see!

Maybe we want to pick a darker color, make the line thicker, and hide the turtle

Note the import statement!

```
import java.awt.Color;
public class Jamal {
   public static void main(String[] args) {
       World world = new World();
       Turtle jamal = new Turtle(world);
        jamal.hide();
       jamal.setColor(Color.BLACK);
       jamal.setPenWidth(20);
       jamal.forward(100);
       jamal.turn(90);
       jamal.forward(100);
       jamal.turn(-45);
       jamal.forward(100);
   }
}
```

The import statement makes Color available.

You don't need to memorize import statements!

But if you forget one, you should recognize the error and be able to fix it (maybe look at an example or Google it)

```
World world = new World();
Turtle jamal = new Turtle(world);
jamal.forward(100);
jamal.turn(90);
jamal.forward(100);
jamal.turn(-45);
jamal.forward(100);
```

How many constructor invocations are in this code?

A. 0
B. 1
C. 2
D. 3
E. 4 or more

```
World world = new World();
Turtle jamal = new Turtle(world);
jamal.forward(100);
jamal.turn(90);
jamal.forward(100);
jamal.turn(-45);
jamal.forward(100);
```

How many constructor invocations are in this code?

A. 0
B. 1
C. 2
D. 3
E. 4 or more

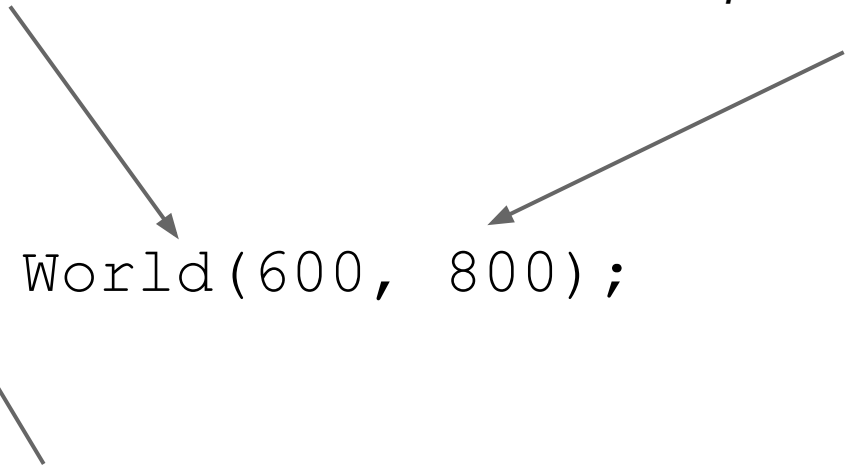
Creating new instances in Java
Constructor Invocation

```
new World(600, 800);
```

Creating new instances in Java

this is the name of the object we are creating

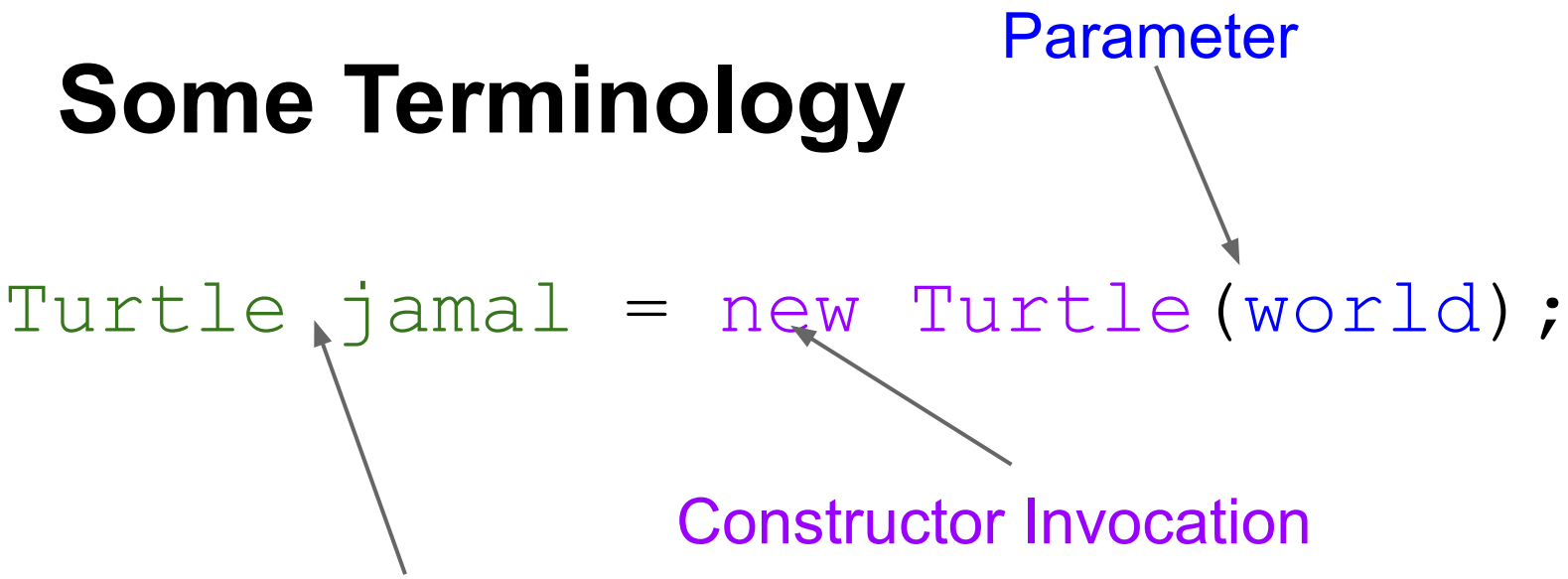
zero or more values inside the parens; called *parameters*



```
new World(600, 800);
```

new is a *keyword* in Java (i.e. a word with a special meaning, like *int* or *double* or *public*)

Some Terminology



```
World world = new World();
Turtle jamal = new Turtle(world);
jamal.forward(100);
jamal.turn(90);
jamal.forward(100);
jamal.turn(-45);
jamal.forward(100);
```

How many non-constructor method invocations are in this code?

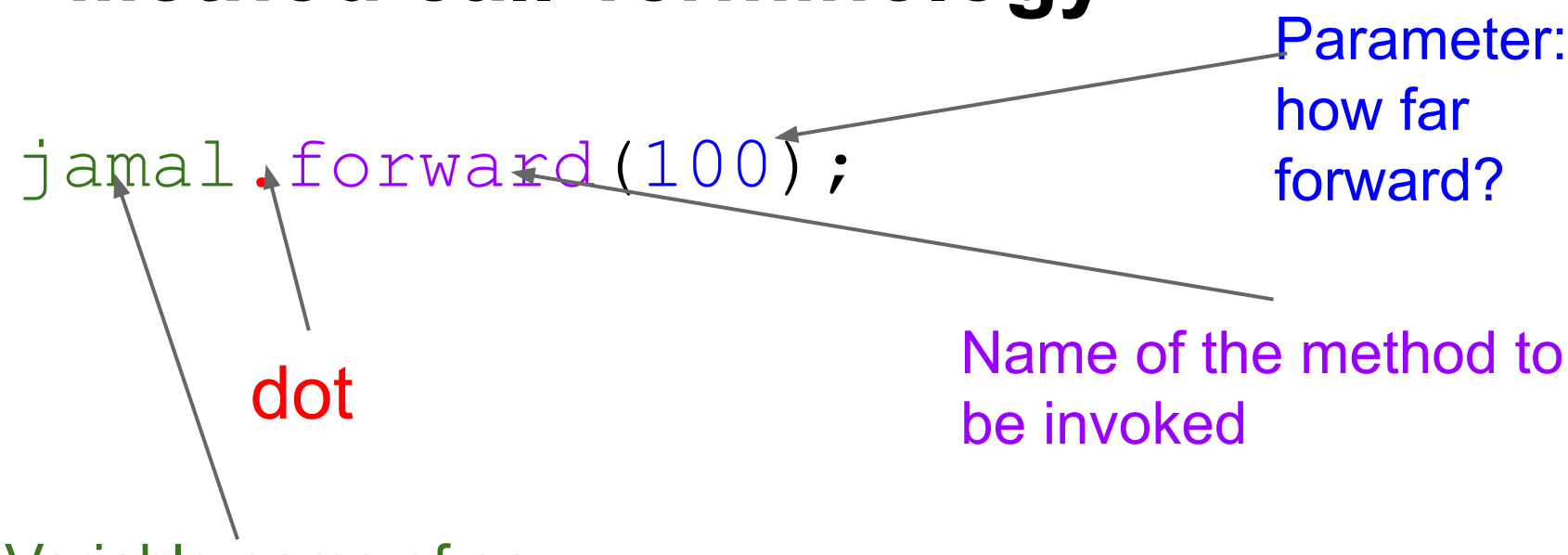
A. 0
B. 1
C. 2
D. 3
E. 4 or more

```
World world = new World();
Turtle jamal = new Turtle(world);
jamal.forward(100);
jamal.turn(90);
jamal.forward(100);
jamal.turn(-45);
jamal.forward(100);
```

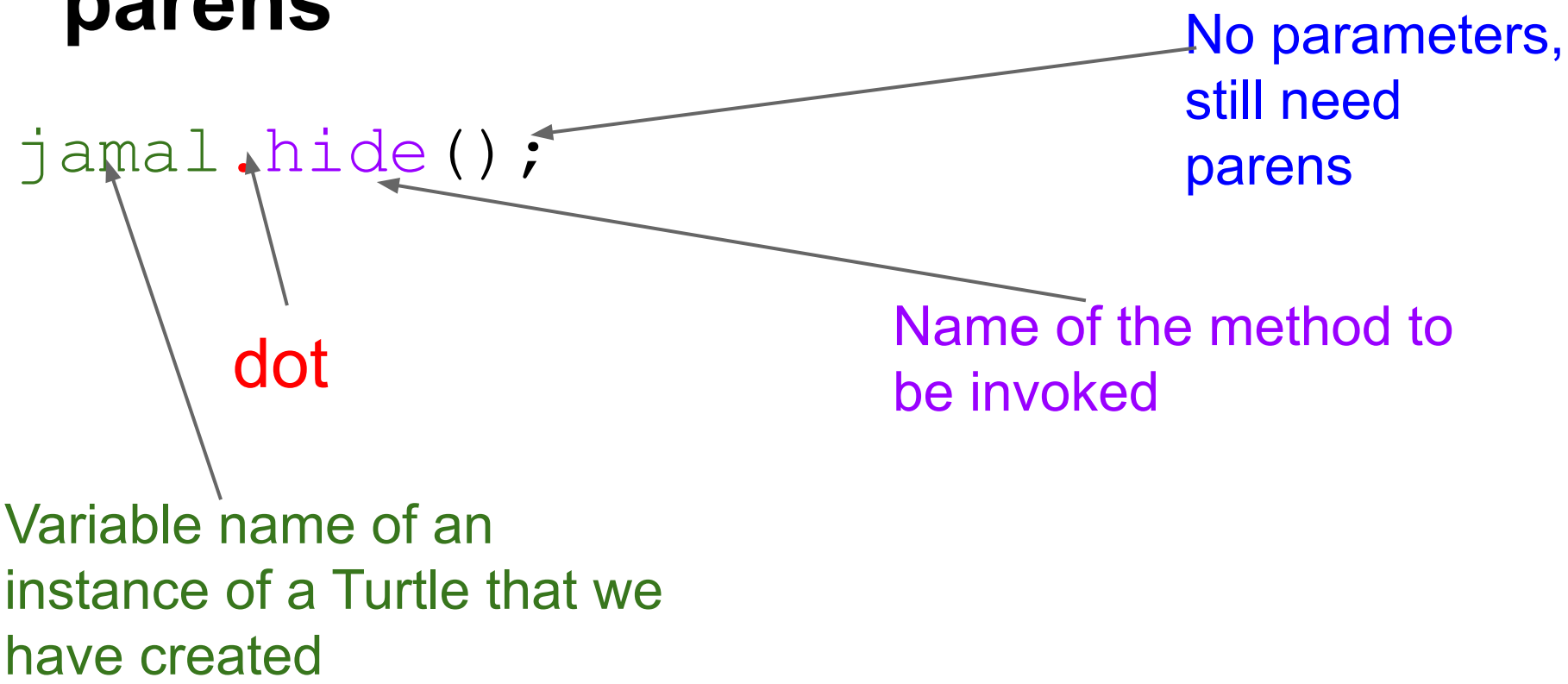
How many non-constructor method invocations are in this code?

A. 0
B. 1
C. 2
D. 3
E. 4 or more

Method call Terminology



Method invocations ALWAYS need parens

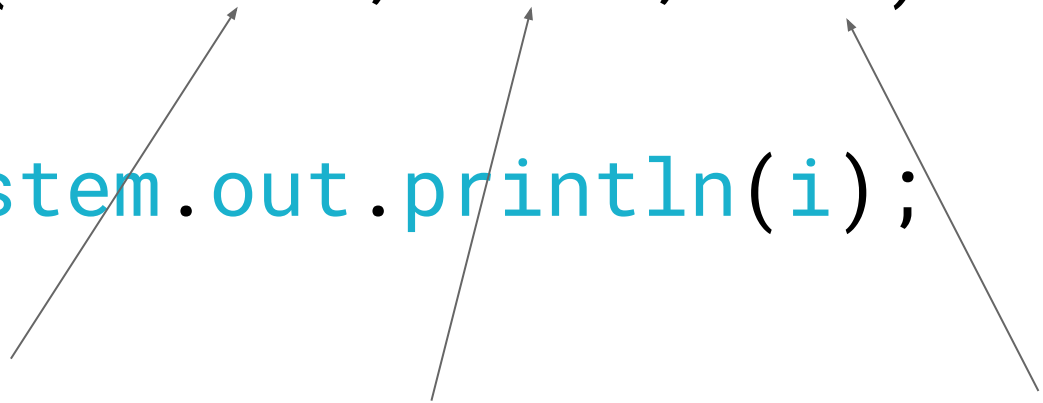


How to repeat things
(We will cover more in Chap. 4)

```
for (int i=0; i<10; i++)
{
   System.out.println(i);
}
```

How to repeat things
(We will cover more in Chap. 4)

```
for (int i=0; i<10; i++)
{
   System.out.println(i);
}
```



start counting at this value

keep going while this is true

change the counter by this much each time

How many repeat something 5 times?

```
for (int i=0; i<5; i++)
for (int i=1; i<5; i++)
for (int i=1; i<=5; i++)
for (int i=0; i<10; i+=2)
```

A 0

B 1

C 2

D 3

E 4

How many repeat something 5 times?

```
for (int i=0; i<5; i++)
for (int i=1; i<5; i++)
for (int i=1; i<=5; i++)
for (int i=0; i<10; i+=2)
```

A 0

B 1

C 2

D 3

E 4

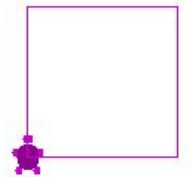
What does this draw?

```
World w = new World();
Turtle carla = new Turtle(w);
for (int i=0; i<4; i++) {
   carla.forward(100);
   carla.turn(-270);
}
```

A. A square!
B. A rectangle!
C. A triangle!
D. Nothing, because it crashes!
E. A structure that cannot be visualized in only 3 dimensions

What does this draw?

```
World w = new World();
Turtle carla = new Turtle(w);
for (int i=0; i<4; i++) {
   carla.forward(100);
   carla.turn(-270);
}
```



A -270 degree turn is the same as a 90 degree turn.

A. A square!

B. A rectangle!

C. A triangle!

D. Nothing, because it crashes!

E. A structure that cannot be visualized in only 3 dimensions

We got this far

If we have extra time, maybe show off more examples of what you can do with Turtles and loops

Turtle Graphics has many other methods

```
void turn(double degrees)
void forward(int distance)
void penUp()
void penDown()
void moveTo(int x, int y)
void setPenColor(Color c)
```

Suppose you want to draw a polygon with a variable number of sides

```
Turtle paige=new Turtle(new World());
int sides=_____;
for (int i=0; i<sides; i++) {
   paige.forward(100);
   paige.turn(_______);
}
```

Suppose we don't know what int value was placed here

What goes in the blank?

A. 360/sides
B. 100/sides
C. 360.0
D. 360.0/sides
E. none of the above

Suppose you want to draw a polygon with a variable number of sides

```
Turtle paige=new Turtle(new World());
int sides=_____;
for (int i=0; i<sides; i++) {
   paige.forward(100);
   paige.turn(_______);
}
```

Suppose we don't know what int value was placed here

The turn() method expects a double. But, anywhere Java expects a double, you can put an int, and it will convert it.

D is the best answer here. **A** only works if the

What goes in the blank?

A. 360/sides
B. 100/sides
C. 360.0
D. 360.0/sides
E. none of the above

Drawing a circle

Try drawing a circle by going forward 1 and turning 1 360 times

HINT: It doesn't work. Why? Rounding!

What can we do instead to approximate a circle?

Suppose you want to draw a circle

```
Turtle casey=new Turtle(new World());
int sides=_____;
int length=_______;
for (int i=0; i<sides; i++) {
   casey.forward(length);
   casey.turn(360/sides);
}
```

What goes in the blanks?

A. 360 360
B. 360 100
C. 1 1
D. 360 1
E. none of the above

Suppose you want to draw a circle

```
Turtle casey=new Turtle(new World());
int sides=_____;
int length=_____;
for (int i=0; i<sides; i++) {
   casey.forward(length);
   casey.turn(360/sides);
}
```

What goes in the blanks?

A. 360 360
B. 360 100
C. 1 1
D. 360 1
E. none of the above

Turtle Graphics methods

Full list of methods (also called the API) is here:

http://cs.knox.edu/BookClassesDocs/index.html?SimpleTurtle.html

Setting pen colors

```
import java.awt.Color;

public class TestTurtle3 {
    public static void main(String[] args) {
        World world=new World();
        Turtle morgan=new Turtle(world);
        morgan.setPenColor(Color.RED);
        morgan.forward(100);
    }
}
```

Must remember to import java.awt.Color!

Imports go before the start of the class

```
import java.awt.Color;


public class TestTurtle3 {
    public static void main(String[] args) {
        World world=new World();
        Turtle morgan=new Turtle(world);
        morgan.setPenColor(Color.RED);
        morgan.forward(100);
    }
}
```

Import statements tell Java that you are going to use certain classes, and that Java should make sure those are available and loaded

You don't need to memorize how to write an import statement; you can look it up…

But, you need to know *what to do if your program does not compile*

Name of the program is line 1

Start of main is line 2

These must be there, and you need to know that they have to be there. But you don't have to memorize this.

```
1: public class TestTurtle {
2:     public static void main(String[] args) {
3:         World world=new World();
4:         Turtle jamal=new Turtle(world);
5:         jamal.forward(100);
6:         jamal.turn(90);
7:         jamal.forward(100);
8:         jamal.turn(-45);
9:         jamal.forward(100);
10:    }
11: }
```

Create a new instance of World.

The world is basically the big flat surface where the Turtle draws everything that it draws

The code is a *constructor invocation*

```
1:  public class TestTurtle {
2:      public static void main(String[] args) {
3:          World world=new World();
4:          Turtle jamal=new Turtle(world);
5:          jamal.forward(100);
6:          jamal.turn(90);
7:          jamal.forward(100);
8:          jamal.turn(-45);
9:          jamal.forward(100);
10:     }
11: }
```

Will this work?

```
Turtle jamal = new Turtle(new World());
```

A: Yes!

not A: No!

Will this work?

```
Turtle jamal=new Turtle(new World());
```

A: Yes!

not A: No!

This will work fine. The Turtle constructor expects an instance of world.

When a method or constructor parameter expects an